

◻ Wizard's Word Scramble ◻

Technical Documentation

Version 1.0 · Last updated June 2026

Table of contents

1. Introduction
2. Architecture overview
3. File reference — index.html
4. File reference — style.css
5. File reference — script.js
6. Core algorithm — Fisher-Yates shuffle
7. Game state & data flow
8. Function reference
9. Extending the game
10. Known limitations & future work

1. Introduction

This document describes the internal design and implementation of Wizard's Word Scramble, a single-page browser game. It is intended for developers who want to understand, modify, or extend the codebase. For installation and play instructions, see the accompanying README.

The project demonstrates two foundational programming concepts in a self-contained way: string manipulation (splitting, rearranging, and joining characters) and randomization (using pseudo-random number generation to shuffle data fairly). It intentionally avoids frameworks or build tooling so the full logic is readable in a single file.

2. Architecture overview

The application follows a simple three-layer separation common to static front-end projects:

Layer	File	Responsibility
Presentation	index.html	DOM structure: spellbook card, input field, buttons, stat panels
Styling	style.css	Visual theme, layout, fonts, animations (starfield, glow effects)
Behavior	script.js	Game state, shuffle algorithm, event handling, DOM updates

There is no backend, database, or persistent storage. All state lives in memory and resets when the page reloads.

3. File reference — index.html

Defines the page skeleton. Key elements and their IDs, which script.js binds to directly:

Element ID	Purpose
#stars	Container that JS populates with animated star divs
#score	Displays current mana (points)
#attempts	Displays number of guesses made
#streak	Displays current correct-answer streak
#scrambled-word	Displays the shuffled letters for the current word
#hint-text	Displays the hint prompt or the revealed clue
#player-input	Text input where the player types their guess
#feedback	Displays correct/incorrect feedback messages

```
#check-btn / #new-word-btn / #hint-btn / #give-up-btn
```

The page also loads two Google Fonts — Cinzel for headings (a serif evoking engraved stone or old manuscripts) and EB Garamond for body text.

4. File reference — style.css

Notable visual techniques used:

- Starfield: absolutely positioned 3px dots with a twinkle keyframe animation, each given a randomized animation-delay in JS so they don't pulse in sync
- Spellbook card: radial-gradient background plus inset/outer box-shadow glow to simulate a magical light source
- Page corners: four small bordered squares positioned at the card's corners to mimic an old manuscript page
- Text glow: text-shadow applied to the title and scrambled word for a luminous effect
- Responsive layout: a single centered flex container with a max-width, so it scales from mobile to desktop without media queries

5. File reference — script.js

All game logic lives in this file. It is organized as a sequence of small, named functions rather than classes, which keeps the control flow easy to trace for beginners.

6. Core algorithm — Fisher-Yates shuffle

The letter-scrambling logic uses the Fisher-Yates shuffle, the standard algorithm for producing an unbiased random permutation of a list:

```
function shuffleWord(word) {  
       let letters = word.split("");  
  
       for (let i = letters.length - 1; i > 0; i--) {  
               let randomIndex = Math.floor(Math.random() * (i + 1));  
               let temp = letters[i];  
               letters[i] = letters[randomIndex];  
               letters[randomIndex] = temp;  
    }  
  
       let scrambled = letters.join("");  
       if (scrambled === word) return shuffleWord(word);  
       return scrambled;  
}
```

Why iterate backwards? Each pass picks a random index from the remaining unshuffled portion (0 to i inclusive) and swaps it into position i. This guarantees every letter has an equal probability of ending up in any position, unlike naive approaches (such as repeatedly picking two random indices to swap a fixed number of times) which can produce biased results.

Why the recursive re-shuffle check? If the shuffled result happens to be identical to the original word (rare, but possible with short words), the function calls itself again so the player is never shown an "unscrambled" word.

7. Game state & data flow

Game state is held in four module-level variables, intentionally simple given the project's scope:

Variable	Type	Purpose
currentWord	string	The correct answer for the active round
score	number	Total mana earned (10 per correct answer)
attempts	number	Total guesses submitted, right or wrong
streak	number	Consecutive correct answers; resets to 0 on a wrong guess

Round lifecycle: `loadNewWord()` picks a random entry from `wordList`, stores it in `currentWord`, scrambles it via `shuffleWord()`, and writes the result to the DOM. A correct guess in `checkGuess()` increments `score` and `streak`, then calls `loadNewWord()` again after a 1.5 second delay (via `setTimeout`) so the player sees the success message before the next round begins.

8. Function reference

Function	Description
<code>createStars()</code>	Generates 60 star elements at randomized x/y positions and animation delays; called once
<code>shuffleWord(word)</code>	Returns a randomly permuted version of the input string using Fisher-Yates
<code>loadNewWord()</code>	Selects a new random word, scrambles it, resets the input and feedback areas
<code>checkGuess()</code>	Compares the player's input (uppercased) against <code>currentWord</code> ; updates score, streak, and
<code>showHint()</code>	Looks up the hint for <code>currentWord</code> in <code>wordList</code> and displays it
<code>giveUp()</code>	Reveals the correct answer, resets streak, and loads a new word after a delay
<code>updateStats()</code>	Writes score, attempts, and streak values into their corresponding DOM elements

9. Extending the game

Add more words

Append objects to the `wordList` array in `script.js`. Each object requires a word (uppercase recommended for consistency) and a hint string.

Add a timer

Introduce a countdown using `setInterval()` alongside the existing `setTimeout()` calls. Decrement a `timeLeft` variable each second, update a new DOM element, and call `giveUp()` automatically when it reaches zero.

Add difficulty levels

Split `wordList` into separate arrays (e.g. `easyWords`, `hardWords`) keyed by word length, and let the player choose a difficulty before `loadNewWord()` picks from the appropriate array.

Persist high scores

The current version keeps no persistent storage. A future version could store the best score using the browser's `localStorage` API, read on page load and written whenever score increases.

10. Known limitations & future work

- No persistent storage — refreshing the page resets score, attempts, and streak to zero
- Word list is fixed in the source code; there is no way to add words through the UI
- No mobile-specific input handling beyond responsive CSS (on-screen keyboard behavior is left to the browser default)

- No automated tests are currently included in the repository
- Single difficulty level — all words are presented with equal probability regardless of length

End of technical documentation.